

Python Weekly Assignment

Topics: OOP · Classes · Encapsulation · Polymorphism · Inheritance · Method Overriding · File Handling · Menu-Driven App

Section A — Classes and Objects

1. Create a class `BankAccount` with the following:

- a) Attributes: `account_holder`, `account_number`, and `balance`.
- b) Methods: `deposit(amount)`, `withdraw(amount)`, and `get_balance()`.
- c) `withdraw()` should print a message and do nothing if the amount exceeds the current balance.
- d) A `__str__` method that returns a readable summary of the account.

Create two `BankAccount` objects and perform a few deposits and withdrawals on each.

2. Create a class `Student` with attributes `name`, `roll_no`, and `marks` (a list of 5 subject marks). Add methods:

- a) `calculate_percentage()` — returns the average of all marks.
- b) `get_grade()` — returns 'A' for ≥ 80 , 'B' for ≥ 60 , 'C' for ≥ 40 , and 'Fail' otherwise.
- c) `display()` — prints all details including percentage and grade.

Create a list of 3 `Student` objects and call `display()` on each.

Section B — Encapsulation

3. Create a class `Employee` with private attributes `__name`, `__emp_id`, and `__salary`. Add:

- a) Getter and setter methods for all three attributes.
- b) The salary setter should raise a `ValueError` if the value is below 10000.
- c) A method `give_raise(percent)` that increases salary by the given percentage.

Demonstrate that the private attributes cannot be accessed directly, and that all access goes through the methods.

4. Build a class `SecureVault` that stores a private `__pin` and a private `__data` dictionary. It should have:

- a) A method `add_entry(pin, key, value)` that adds to `__data` only if the pin matches.
- b) A method `get_entry(pin, key)` that returns the value only if the pin matches.
- c) Any wrong pin attempt should print 'Access Denied' and return nothing.

Section C — Inheritance and Method Overriding

5. Create a base class `Shape` with a method `area()` that returns 0 and a method `describe()` that prints 'I am a shape'. Create three subclasses:

- a) `Circle(radius)` — overrides `area()` using $\pi * r^2$.
- b) `Rectangle(length, width)` — overrides `area()`.
- c) `Triangle(base, height)` — overrides `area()`.

Each subclass should also override `describe()` to print something specific about itself. Create one object of each and call both methods.

6. Create a class `Vehicle` with attributes `make`, `model`, and `year` and a method `start_engine()` that prints 'Engine started'. Create two subclasses:

- a) `Car(make, model, year, num_doors)` — overrides `start_engine()` to include the car's name in the message.
- b) `ElectricCar(make, model, year, battery_capacity)` — overrides `start_engine()` to print 'Silent start — battery at X kWh'.

Use `super()` in both subclasses to call the parent `__init__`. Create objects and test all methods.

Section D — Polymorphism

7. Create a base class `Animal` with a method `sound()` that prints 'Some sound'. Create subclasses `Dog`, `Cat`, and `Cow`, each overriding `sound()` with their respective sounds. Write a function `make_all_sounds(animals)` that takes a list of `Animal` objects and calls `sound()` on each. Demonstrate that the correct method is called regardless of the object type.

8. Create a class `Calculator` with overloaded-style methods using default arguments:

- a) `add(a, b, c=0)` — adds two or three numbers.
- b) `multiply(a, b, c=1)` — multiplies two or three numbers.
- c) Also implement `__add__` on a class `Vector(x, y)` so that two `Vector` objects can be added with the `+` operator. Print the resulting vector.

Section E — File Handling

9. Perform the following file operations. Use plain text files only.

- a) Write a list of 5 student names to a file `students.txt`, one name per line.
- b) Read the file back and print each name with its line number.
- c) Append two more names to the same file without overwriting existing content.
- d) Read the final file and count the total number of students.
- e) Copy the contents of `students.txt` into a new file `students_backup.txt`.

Section F — Menu-Driven App with OOP

10. Build a menu-driven Student Record Manager using OOP and file handling. The app should have a class `StudentManager` with the following menu options:

- 1. Add Student — takes name, roll number, and marks for 3 subjects.
- 2. View All — displays all stored students with their average marks.
- 3. Search Student — finds a student by roll number.
- 4. Delete Student — removes a student by roll number.
- 5. Save to File — writes all records to `students_data.txt`.
- 6. Load from File — reads records back from `students_data.txt` on startup.
- 7. Exit

Requirements: use a class with a list as internal storage, handle invalid menu choices gracefully, and use a while loop to keep the menu running until the user chooses to exit.